

Reference material on OpenMP and PDC concepts for after week 14

Review of PDC to this point in the course

We have seen an example of a simple use of *data parallelization* to parallelize a 'for each' loop that did the same operation on each element of a vector or other data structure in [this assignment](#). As a reminder this involved adding the `#pragma omp parallel` directive before the loop. This directive caused the loop to be separated into different chunks that were able to be done at the same time by different processors.

We have also started to consider the idea of *task parallelism* in which different tasks can be performed at the same time. The *Fork-Join* idea is that different tasks can be performed at the same time, but it is also important to make sure that a task that depends on a prior task is forced to wait, requiring *synchronization*. Remember [this assignment](#) from week 5.

This week it is time to expand on task parallelism as it applies to sorting algorithms and try out some other openMP compiler directives.

Vocabulary

Thread: A thread is like a mini-program running inside your program. Each thread runs part of the code. Threads can be run in parallel on different processors. Threads by default share the same memory.

Processor: The physical hardware that executes the instructions.

Features of OpenMP

Parallel region: Code blocks marked with `#pragma omp parallel` define a parallel region and the blocks are executed by multiple threads.

Data sharing: Variables can be shared among threads or private to each thread.

Synchronization: OpenMP provides mechanisms for synchronizing threads.

Thread Management: OpenMP manages the creation and management of threads.

OpenMP basic syntax

```
#include <omp.h>
```

```
int main() {
```

```

        #pragma omp parallel # defines a parallel region
        { // this curly brace must be on it's own line and can't be on
the same line as the directive
            // code to be executed in parallel by multiple threads
            int thread_id = omp_get_thread_num(); // returns the
unique id of the thread currently executing the code
        }
    }

```

You can set the number of threads used by a program:

```

#include <omp.h>
#include <thread> // can be used to find out how many processors

int main() {

    // using thread to find out how many processors the current
computer has
    const auto processor_count = thread::hardware_concurrency();
    int num_threads = 10;

    // By default OpenMP may adjust the number of threads at
runtime to
    // optimize performance based on system load and resources. But
you can have
    // full control by using the following command
    omp_set_dynamic(false);

    omp_set_num_threads(num_threads); // set the number of threads
to 10
}

```

Open MP and Task Parallelism

Here are two different ways that OpenMP can be used for Task Parallelism:

- (1) `#pragma omp parallel sections` is used when you have independent tasks that can run in parallel.

Here's an example of two tasks A, and B that will be run in parallel.

```

int main() {
    #pragma omp parallel sections
    {

```

```

        #pragma omp section
        {
            taskA();
        }
        #pragma omp section
        {
            taskB();
        }
    }
}

```

(2) Use `#pragma omp parallel` and `#pragma omp task`

```

int main() {
    #pragma omp parallel
    {
        #pragma omp task
        {
            taskA();
        }
        #pragma omp task
        {
            taskB();
        }
        #pragma omp taskwait // stop and wait for both A and B to
be done
    }
}

```